

When Simple Wins: Lightweight CNN Encoders for Resource-Constrained Nucleus Segmentation

Danny Rollo

Northeastern University, Boston MA, USA
rollo.1@northeastern.edu

Abstract. For nucleus segmentation pipelines built without large annotated datasets or high-memory GPU infrastructure, a simple VGG U-Net encoder matches purpose-built transformer alternatives at a fraction of Swin-T’s parameter count. We present a controlled comparison of VGG-style, ResNet-style, and Swin Transformer encoders within a shared U-Net framework, evaluated on PanNuke (~5K training patches, 19 tissue types) and MoNuSeg (37 training whole-slide images). On PanNuke, VGG (Dice 0.854) matches ResNet (0.853) to within 0.002 Dice and outperforms pretrained Swin by 3.0 Dice points (0.824). On MoNuSeg, VGG (0.794) reaches published MedT-level performance—a transformer designed specifically for small medical datasets—while outperforming pretrained Swin by 3.6 Dice points (0.758). VGG is also the fastest and most memory-efficient of the learned models we test, making it Pareto-dominant over Swin-T at inference. Lightweight from-scratch architectures remain the practical choice for edge microscopy, veterinary and rare-disease research, and non-H&E protocols. Of the three encoders we compare, a simple VGG is the strongest.

Keywords: Nucleus segmentation · Histopathology · U-Net · Lightweight models · Low-data.

1 Introduction

Nucleus segmentation in histopathology images underlies a broad range of clinical and research workflows, from tumor microenvironment quantification to cell-cycle analysis. Computational pathology increasingly scales transformer-based foundation models to hundreds of millions of parameters pretrained on millions of whole-slide images [4,25].

This eases the labeled-data shortage but constrains deployment in ways recent work has documented: foundation models are typically restricted to linear probing rather than full fine-tuning due to memory and stability constraints [23,3]; their robustness to cross-site distribution shift has been questioned [5]; and their H&E-pretrained features transfer poorly to non-standard stains such as IHC [13]. For deployment scenarios such as edge-deployed microscopy in under-served settings [2] and veterinary or rare-disease research [27], where annotation budgets permit only tens of whole-slide images, lightweight from-scratch architectures remain the practical choice.

This raises a concrete design question: within the space of lightweight, from-scratch architectures, which encoder is best in the low-data regime? We answer it with a controlled ablation comparing VGG, ResNet, and Swin encoders on PanNuke and the genuinely small-data MoNuSeg (37 images), holding the decoder and encoder interface fixed so the backbone is the only variable. A VGG encoder outperforms both ResNet and pretrained Swin on PanNuke and matches MedT (a transformer built for small medical datasets) while outperforming pretrained Swin on MoNuSeg.

While this resolution-vs-inductive-bias tradeoff is qualitatively well understood, we are not aware of prior work isolating encoder family as the sole variable across both a patch-scale and a genuinely small whole-slide-image benchmark, attributing the performance gap to architecture rather than to the recipe, data, and hyperparameter confounds that limit cross-paper comparisons.

Contributions. (1) A controlled encoder-family ablation within a shared U-Net framework on PanNuke and MoNuSeg; (2) evidence that a from-scratch VGG encoder matches MedT (a transformer purpose-built for small medical datasets) on MoNuSeg while beating pretrained Swin on both; (3) inference-cost measurements showing VGG is Pareto-dominant over Swin-T in the lightweight regime.

Code and dependencies. We implement the U-Net, training loop, evaluation, and data loaders from scratch in PyTorch (`torchvision` for I/O and augmentation); the Swin encoder loads a `timm` [26] backbone with pretrained ImageNet weights, PanNuke comes from HuggingFace `datasets`, MoNuSeg from the MICCAI 2018 challenge, and Otsu from `scikit-image`. Code, configurations, trained checkpoints, and exact dependency versions are available at <https://github.com/Bedrockdude10/BinaryCellSegmentation>.

2 Related Work

U-Net [19] introduced the symmetric encoder-decoder whose skip connections concatenate high-resolution encoder features into the decoder for precise localization; demonstrated on limited biomedical data, it remains the dominant medical segmentation backbone.

Residual Networks [11] addressed degradation in deep networks via identity shortcuts; He et al. [10] separately proposed MSRA initialization for ReLU networks. Our ablation directly compares VGG- and ResNet-style encoders.

Vision Transformers [6] match CNNs on classification but require large-scale pretraining to compensate for the absence of convolutional inductive biases. The Swin Transformer [16] adds shifted-window self-attention, producing hierarchical feature maps suited to dense prediction.

Pathology Foundation Models. UNI [4] (307M parameters, 100K slides) and Virchow2 [25] (632M parameters, 1.5M slides) typify large-scale pathology pre-training. A clinical benchmark [3] finds their gains concentrate on slide-level classification with more modest segmentation gains; Tizhoosh [23] documents heavy compute, poor robustness, and safety concerns; De Jong et al. [5] show they

encode medical-center identity, limiting cross-site generalization; and PathoDuet [13] shows H&E features need cross-stain training for IHC—motivating lightweight alternatives.

MedT [24], our most directly relevant baseline, proposed gated axial-attention with a local-global (LoGo) training strategy for the small-medical-dataset regime. It reported Dice 0.7955 and IoU 0.6617 on the MoNuSeg test set (14 images), trained on 30 whole-slide images from the Kumar 2017 release [15]; we train on the 2018 expansion (37 images, +7).

Instance-aware methods. HoVer-Net [9] predicts distance maps to separate overlapping nuclei beyond binary masks; HoVer-NeXt [1] modernizes it with ConvNeXt-V2 backbones, and CellViT [12] uses a histology-pretrained ViT. These run at higher complexity and larger data budgets than our target regime. We also exclude the widely used StarDist [20] and Cellpose [22]: their star-convex-polygon and flow-field targets, rather than binary masks, would introduce an annotation-format confound that conflicts with our single-variable design.

3 Methods

Our methodological contribution is a controlled ablation: we fix the decoder, training recipe, data pipeline, and evaluation protocol, varying only the encoder backbone across three representative families (VGG, ResNet, Swin), which isolates encoder choice as the single source of performance variation.

3.1 Intuition

We expect CNNs to beat Swin in this low-data regime for three reasons. (i) *Skip-connection resolution*: nucleus boundaries are a few pixels wide, so the decoder needs high-resolution features; VGG preserves full 256×256 resolution at its first skip, whereas Swin’s 4×4 tokenization downsamples $4\times$ immediately and bilinear upsampling cannot recover the detail. (ii) *Inductive bias vs. data scale*: self-attention needs more data than convolution to learn translation-equivariance and local-structure priors—unlikely at $\sim 5\text{K}$ PanNuke patches or 37 MoNuSeg images, even with ImageNet pretraining (an H&E domain mismatch). (iii) *Input scale*: on 256×256 patches a stacked- 3×3 VGG receptive field already covers most of the input by stage 4, limiting global attention’s value. Section 4 tests these, including whether residual connections or loss choice matter at four stages.

3.2 Architecture

Our pipeline uses a U-Net with a fixed decoder and a shared encoder interface: each encoder’s `forward(x)` returns four feature maps at progressively lower resolutions and exposes per-stage channel counts, which the decoder reads dynamically—so swapping backbones requires only configuration changes.

Decoder. The decoder is a bottleneck block followed by four upsampling stages, each a transposed convolution, concatenation with the matching encoder skip connection, and a two-layer conv block (batch normalization [14] and ReLU); a final 1×1 convolution produces single-channel logits. Batch normalization follows every convolution in all encoders.

VGG Encoder. The VGG encoder follows the design principles of [21]: four stages of stacked 3×3 convolutions with batch normalization and ReLU, with max-pooling between stages for spatial downsampling. Channel widths double at each stage: $64 \rightarrow 128 \rightarrow 256 \rightarrow 512$. Features are stored *before* pooling to preserve full spatial resolution for skip connections, producing outputs at 256^2 , 128^2 , 64^2 , and 32^2 for 256×256 inputs. Total parameters: $\sim 34\text{M}$.

ResNet Encoder. The ResNet encoder replaces plain convolutional blocks with residual blocks [11]. A lightweight 3×3 stem (stride 1) replaces the standard ResNet stem to match VGG’s spatial dimensions. Downsampling in stages 2–4 uses stride-2 convolution in the first block of each stage, with 1×1 projection shortcuts. Channel widths match VGG. Total parameters: $\sim 37.5\text{M}$.

Swin Transformer Encoder. The Swin encoder wraps a Swin-T backbone [16] via `timm`, with channels $96 \rightarrow 192 \rightarrow 384 \rightarrow 768$. Its 4×4 tokenization natively yields features at $H/4$, $H/8$, $H/16$, $H/32$ —lower resolution than the CNN encoders at every stage—which we bilinearly upsample to 256^2 , 128^2 , 64^2 , 32^2 . We evaluate randomly initialized and ImageNet-pretrained variants. Total parameters: $\sim 87\text{M}$.

3.3 Loss Functions

We compare two losses on sigmoid-activated logits: per-pixel Binary Cross-Entropy (BCE) and region-based Dice loss, which directly optimizes the Dice coefficient and handles class imbalance:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)], \quad \mathcal{L}_{\text{Dice}} = 1 - \frac{2 \sum_i \hat{y}_i y_i + \epsilon}{\sum_i \hat{y}_i + \sum_i y_i + \epsilon}.$$

3.4 Classical Baseline

Otsu thresholding [18] is the non-learned floor: each test image is converted to grayscale and thresholded at the intensity maximizing between-class variance, classifying darker pixels as nuclei (nuclei appear darker than background in H&E). It has zero learnable parameters and requires no training.

3.5 Datasets

Data use declaration. Both datasets are publicly available and released under the CC BY-NC-SA 4.0 license; they comprise de-identified, previously published images, so no additional ethics approval was required for this work.

PanNuke [7,8] provides 7,904 256×256 patches across 19 tissue types with 189,744 nuclei in 5 categories, pre-split into three folds. We train on fold1, validate on fold2, and evaluate on fold3; binary masks collapse all cell-type labels to foreground. Augmentation is random flips and 90° rotations with ImageNet normalization.

MoNuSeg [15] provides 37 training whole-slide images (1000×1000 , H&E, XML polygon annotations over 7 organs) and 14 test images whose lung and brain organs are absent from training, testing generalization to unseen tissue. We use the 2018 expansion.¹ Images are reflection-padded to 1024×1024 and tiled into 16 non-overlapping 256×256 patches; validation is carved at the image level (20%, seed 42) so no slide spans both splits. Test images are tiled identically, with per-image Dice averaged over 16 patches.

3.6 Training Details

All models train with SGD (momentum 0.9, weight decay 10^{-4} , batch size 8), with early stopping on validation Dice and consistent augmentation and ImageNet normalization. CNN encoders use learning rate 0.01 with step decay ($\gamma = 0.1$ at epoch 30); pretrained Swin uses 10^{-4} with cosine annealing. On **PanNuke**, patience is 15 (CNNs converge in 48–50 epochs); on **MoNuSeg**, patience is 20 with a 100-epoch cap (CNN) or 150 (Swin).

4 Experiments and Results

Testbed. All results run on a single NVIDIA Tesla V100-SXM2-32GB in fp32: accuracy averaged over 5 training seeds, inference cost (Table 3) measured separately on an idle GPU. Training uses batch size 8 and the recipe of Section 3.6, with data pipeline, augmentation, optimizer, and decoder fixed so that only the encoder varies. Metrics are Dice and IoU on the held-out splits (PanNuke fold3; MoNuSeg 14-image test set).

Research questions. We address five questions: encoder family (Q1) and the effect of ImageNet pretraining (Q2) on PanNuke; whether the CNN advantage holds in the low-data regime and a VGG encoder reaches MedT’s published performance (Q3) on MoNuSeg; whether residual shortcuts or Dice loss help at four encoder stages (Q4); and whether inference cost matches the lightweight-deployment framing (Q5).

¹ The public challenge description lists 30 training images (the original 2017 release); the training set distributed for the 2018 challenge, which we use, contains 37. MedT reports the 30-image count.

Table 1. Test results on PanNuke fold3, mean $\pm$ standard deviation over 5 training seeds ($n=5$; the Otsu baseline is deterministic). Epochs is the early-stopping epoch. Best result in **bold**.

Encoder	Loss	Dice $\uparrow$	IoU $\uparrow$	Epochs	Params
Otsu (classical)	—	0.482	0.349	—	0
Swin-T (scratch)	BCE	0.825 (0.001)	0.703 (0.002)	69.0 (9.4)	86.8M
Swin-T (pretrained)	BCE	0.824 (0.001)	0.701 (0.001)	96.2 (5.0)	86.8M
ResNet	BCE	0.853 (0.000)	0.743 (0.001)	49.8 (3.8)	37.5M
VGG	Dice	0.856 (0.001)	0.748 (0.001)	84.8 (17.5)	34.0M
VGG	BCE	0.854 (0.000)	0.746 (0.000)	51.8 (2.6)	34.0M

4.1 PanNuke Results

Table 1 summarizes results on PanNuke fold3; all learned models substantially exceed the classical Otsu baseline. VGG and ResNet produce nearly identical results (0.854 vs. 0.853 Dice): the paired difference is 0.002 Dice (95% CI [0.001, 0.002]), reproducible across seeds but an order of magnitude below the ~ 0.005 Dice we treat as practically meaningful, so we do not rank them. Addressing Q1–Q2: Swin underperforms both CNN encoders by roughly 3 Dice points, with or without ImageNet pretraining. This 0.030 Dice gap (paired, 95% CI [0.029, 0.031]) is an order of magnitude larger than the BCE-vs-Dice variation for a fixed encoder (0.002 Dice), indicating a genuine architectural effect rather than seed noise.

4.2 MoNuSeg Results

Table 2 shows MoNuSeg test results (14 images, per-image averaged Dice), answering Q3. VGG reaches Dice 0.794 ± 0.011 , close to MedT’s published 0.796 but not comparable to it on equal terms, and outperforms pretrained Swin (0.758 ± 0.007) by 0.036 Dice, 95% CI [0.021, 0.052]. The MedT comparison is not strictly controlled (different training-set size and input protocol; see Limitations), so we read it only as VGG *reaching* MedT-level Dice. That caveat is confined to this cross-paper anchor: VGG, ResNet, and Swin share the identical 37-image set and protocol, so our controlled VGG-over-Swin result is unaffected; the gap is comparable at both data scales.

4.3 Residual Connections and Loss (Q4)

VGG and ResNet produce nearly identical PanNuke results (0.854 vs. 0.853 Dice): residual connections matter mainly in *deep* networks [11], and at four stages neither suffers gradient degradation, so shortcuts add no benefit worth having—we recommend VGG for its simpler implementation. BCE and Dice loss are likewise near-identical (0.854 vs. 0.856 Dice; 0.746 vs. 0.748 IoU). Both gaps fall below the 0.005 Dice we treat as material, so we rank neither. With moderate

Table 2. Test results on MoNuSeg (14 test images). Dice computed per image, averaged over the test set. MedT results are published figures (30 training images, 512×512 input); ours use 37 training images, 256×256 patches, reported as the mean over 5 training seeds with the standard deviation in parentheses ($n=5$). Best result in **bold**.

Model	Dice $\uparrow$	IoU $\uparrow$	Params
MedT [24] (published, 30 imgs)	0.796	0.662	1.4M
Swin-T (pretrained, ours, 37 imgs)	0.758 (0.007)	0.613 (0.008)	86.8M
VGG (ours, 37 imgs)	0.794 (0.011)	0.660 (0.015)	34.0M

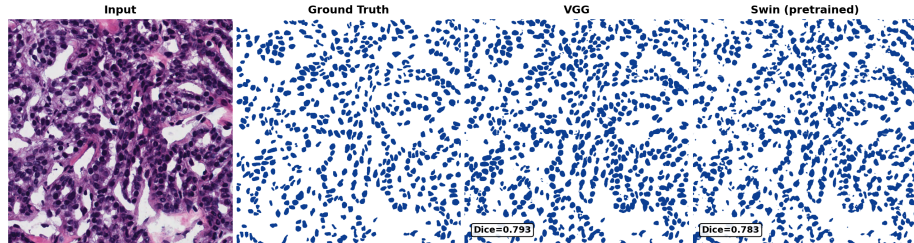


Fig. 1. Qualitative comparison on a representative dense MoNuSeg test region. Left to right: input H&E patch, ground-truth nucleus overlay (green), VGG prediction, and pretrained Swin prediction (per-image Dice annotated in each prediction cell). VGG delineates nuclear boundaries more tightly; Swin produces smoother masks with less precise boundaries.

foreground density BCE is not hurt by class imbalance, and Dice helps only with very sparse foreground.

4.4 Inference Cost

Table 3 reports inference cost on our testbed (median forward-pass latency over 300 iterations on a single 256×256 input after 50 warmup iterations; peak memory via `torch.cuda.max_memory_allocated`), answering Q5. These support the lightweight-deployment framing rather than serving as optimized benchmarks (eager-mode PyTorch, no fusion or quantization).

Swin uses roughly $2.3\times$ the memory and $2.6\times$ the latency of VGG for no accuracy gain. A VGG U-Net fits comfortably on a consumer laptop GPU; even Swin-T already costs meaningfully more.

4.5 Qualitative Analysis

Qualitatively (Fig. 1), Swin’s 4×4 tokenization and bilinear upsampling yield smoother, less precise boundaries than the CNN encoders; all models struggle with dense overlapping clusters that binary masks cannot resolve into instances.

Table 3. Single-image inference cost at 256×256 , batch size 1, fp32, on one idle NVIDIA Tesla V100-SXM2-32GB. Latency is the median of 300 synchronised forward passes after 50 warmup iterations; peak memory covers weights and activations. VGG is the fastest and most memory-efficient learned model.

Model	Params	Latency (ms)	Peak Memory (MB)
VGG U-Net	34.0M	7.1	264.9
ResNet U-Net	37.5M	9.6	277.2
Swin-T U-Net (pretrained)	86.8M	18.4	618.8

5 Discussion

Implications for resource-constrained deployment. Every configuration we train fits a single 8GB consumer GPU: peak training memory was 3.6 GiB for VGG and 5.4 GiB for pretrained Swin-T, the heaviest we ran. A VGG U-Net (~ 34 M parameters) trains overnight with no external data, directly actionable for the edge-microscopy, veterinary, and non-H&E settings [27,17] that motivate this work.

The role of pretraining. Pretraining does not fail on Swin—it closes most of the Otsu-to-CNN gap ($+0.212$ Dice)—but cannot close the final 0.049 to VGG, which we attribute to the skip-connection resolution bottleneck (Section 3.1): a property of the architecture, not the weights. This predicts that even domain-matched pretraining (e.g., UNI [4]) would narrow but not close the gap—a falsifiable target for future work.

5.1 Limitations

Several limitations bound these conclusions. Tables 1 and 2 report mean and standard deviation over five training seeds on one fixed split, and every difference we claim has a paired 95% confidence interval excluding zero; this bounds training-seed variance, not split variance, which with 37 MoNuSeg images is plausibly larger. We give intervals rather than non-parametric tests because at $n=5$ a two-sided signed-rank p cannot fall below 0.0625. We did not sweep learning rates, so cannot quantify the comparison’s sensitivity to them; from-scratch Swin’s instability at 10^{-2} suggests it is not negligible. Pretrained Swin reached its 100-epoch cap in two of five seeds, so its schedule may bound it, though those two seeds scored no lower than the three that stopped on patience. We study only binary segmentation, not instance separation of touching nuclei; the MedT comparison is uncontrolled in training-set size (37 vs. 30) and protocol (256×256 tiles vs. one 512×512 pass); and all data are H&E, leaving the non-H&E stains (IHC, fluorescence) that most motivate this work untested.

6 Conclusion

We compared CNN and transformer encoders for nucleus segmentation under real-world resource constraints. In a shared U-Net on PanNuke and MoNuSeg, a from-scratch VGG encoder (34M parameters) outperforms pretrained Swin on both and reaches published MedT-level performance on MoNuSeg (0.794 vs. 0.796 Dice, a rough equivalence given protocol differences)—a margin consistent across both data scales and matching our hypothesized skip-connection-resolution mechanism. For pipelines with tens of annotated images and single-GPU infrastructure, a VGG U-Net encoder is thus the strongest of the three encoders we compare; transformers need large-scale pretraining or far more data to match it.

Future work. Promising directions: a resolution-matched Swin, or a CNN with equally reduced skip resolution, to separate tokenization from encoder family; pretraining matched to H&E rather than ImageNet; the data scale at which pretrained Swin matches VGG; hybrid CNN-transformer encoders; and memory-matched comparison with foundation models (UNI, Virchow2).

References

1. Baumann, E., Dislich, B., Rumberger, J.L., Nagtegaal, I.D., Martínez, M.R., Zlobec, I.: HoVer-NeXt: a fast nuclei segmentation and classification pipeline for next generation histopathology. In: Medical Imaging with Deep Learning (MIDL). PMLR, vol. 250, pp. 61–86 (2024)
2. Bueno, G., Sánchez-Vargas, L., Díaz-Maroto, A., Ruiz-Santaquiteria, J., Blanco, M., Salido, J., Cristóbal, G.: Real-time edge computing vs. GPU-accelerated pipelines for low-cost microscopy applications. *Electronics* **14**(5), 930 (2025)
3. Campanella, G., Chen, S., et al.: A clinical benchmark of public self-supervised pathology foundation models. *Nat. Commun.* **16**, 3640 (2025)
4. Chen, R.J., Ding, T., Lu, M.Y., Williamson, D.F.K., et al.: Towards a general-purpose foundation model for computational pathology. *Nat. Med.* **30**(3), 850–862 (2024)
5. de Jong, E.D., et al.: Current pathology foundation models are unrobust to medical center differences. *arXiv:2501.18055* (2025)
6. Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., Houlsby, N.: An image is worth 16x16 words: transformers for image recognition at scale. In: International Conference on Learning Representations (ICLR) (2021)
7. Gamper, J., Koohbanani, N.A., Benet, K., Khuram, S.A., Rajpoot, N.: PanNuke: an open pan-cancer histology dataset for nuclei instance segmentation and classification. In: European Congress on Digital Pathology (ECDP), pp. 11–19 (2019)
8. Gamper, J., Koohbanani, N.A., Graham, S., Jahanifar, M., Khuram, S.A., Azam, A., Hewitt, K., Rajpoot, N.: PanNuke dataset extension, insights and baselines. *arXiv:2003.10778* (2020)
9. Graham, S., Vu, Q.D., Raza, S.E.A., Azam, A., Tsang, Y.W., Kwak, J.T., Rajpoot, N.: HoVer-Net: simultaneous segmentation and classification of nuclei in multi-tissue histology images. *Med. Image Anal.* **58**, 101563 (2019)

10. He, K., Zhang, X., Ren, S., Sun, J.: Delving deep into rectifiers: surpassing human-level performance on ImageNet classification. In: IEEE International Conference on Computer Vision (ICCV), pp. 1026–1034 (2015)
11. He, K., Zhang, X., Ren, S., Sun, J.: Deep residual learning for image recognition. In: IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp. 770–778 (2016)
12. Hörst, F., Rempe, M., Heine, L., et al.: CellViT: vision transformers for precise cell segmentation and classification. *Med. Image Anal.* **94**, 103143 (2024)
13. Hua, S., Yan, F., Shen, T., Ma, L., Zhang, X.: PathoDuet: foundation models for pathological slide analysis of H&E and IHC stains. *arXiv:2312.09894* (2024)
14. Ioffe, S., Szegedy, C.: Batch normalization: accelerating deep network training by reducing internal covariate shift. In: International Conference on Machine Learning (ICML), pp. 448–456 (2015)
15. Kumar, N., Verma, R., Sharma, S., Bhargava, S., Vahadane, A., Sethi, A.: A dataset and a technique for generalized nuclear segmentation for computational pathology. *IEEE Trans. Med. Imaging* **36**(7), 1550–1560 (2017)
16. Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., Guo, B.: Swin Transformer: hierarchical vision transformer using shifted windows. In: IEEE International Conference on Computer Vision (ICCV), pp. 10012–10022 (2021)
17. Neal, S.V., et al.: Artificial intelligence in veterinary clinical pathology—an introduction and review. *Vet. Clin. Pathol.* (2025)
18. Otsu, N.: A threshold selection method from gray-level histograms. *IEEE Trans. Syst. Man Cybern.* **9**(1), 62–66 (1979)
19. Ronneberger, O., Fischer, P., Brox, T.: U-Net: convolutional networks for biomedical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI), pp. 234–241 (2015)
20. Schmidt, U., Weigert, M., Broaddus, C., Myers, G.: Cell detection with star-convex polygons. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI), pp. 265–273 (2018)
21. Simonyan, K., Zisserman, A.: Very deep convolutional networks for large-scale image recognition. *arXiv:1409.1556* (2014)
22. Stringer, C., Wang, T., Michaelos, M., Pachitariu, M.: Cellpose: a generalist algorithm for cellular segmentation. *Nat. Methods* **18**, 100–106 (2021)
23. Tizhoosh, H.R.: Why foundation models in pathology are failing. *arXiv:2510.23807* (2025)
24. Valanarasu, J.M.J., Oza, P., Hacıhaliloglu, I., Patel, V.M.: Medical Transformer: gated axial-attention for medical image segmentation. In: Medical Image Computing and Computer-Assisted Intervention (MICCAI), pp. 36–46 (2021)
25. Vorontsov, E., Bozkurt, A., Casson, A., et al.: A foundation model for clinical-grade computational pathology and rare cancers detection. *Nat. Med.* **30**(10), 2924–2935 (2024)
26. Wightman, R.: PyTorch Image Models. <https://github.com/huggingface/pytorch-image-models> (2019)
27. Xiao, S., Dhand, N.K., Wang, Z., Hu, K., Thomson, P.C., House, J.K., Khatkar, M.S.: Review of applications of deep learning in veterinary diagnostics and animal health. *Front. Vet. Sci.* **12**, 1511522 (2025)